System Verification and Validation Plan for Lake

Team #10, CinqGarçons
Ethan Walsh
Colin Chambachan
Matthew Nesbitt
Jinwoo Hong
Gary Qin

November 3, 2026

Revision History

Date	Developer(s)	Change
November 2, 2026	All team members	Initial revision

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 2
- 2.4 Relevant Documentation . . . 2

3 Plan **3**
- 3.1 Verification and Validation Team . . . 3
- 3.2 SRS Verification . . . 5
- 3.3 Design Verification . . . 6
- 3.4 Verification and Validation Plan Verification . . . 6
- 3.5 Implementation Verification . . . 7
- 3.6 Automated Testing and Verification Tools . . . 8
- 3.7 Software Validation . . . 9

4 System Tests **10**
- 4.1 Tests for Functional Requirements . . . 10
  - 4.1.1 Coordinator (S.2.1) . . . 11
  - 4.1.2 Model Partitioner (S.2.2) . . . 12
  - 4.1.3 Inference Engine (S.2.3) . . . 13
  - 4.1.4 Tensor I/O (S.2.4) . . . 13
  - 4.1.5 Lake Client (S.2.5) . . . 14
- 4.2 Tests for Nonfunctional Requirements . . . 15
  - 4.2.1 Usability . . . 15
  - 4.2.2 Performance . . . 16
  - 4.2.3 Look and Feel . . . 18
  - 4.2.4 Fault Tolerance . . . 19
  - 4.2.5 Privacy . . . 20
  - 4.2.6 Compatibility . . . 21
  - 4.2.7 Environment and Licensing . . . 23
- 4.3 Tests for Safety Requirements . . . 24
- 4.4 Traceability Between Test Cases and Requirements . . . 27

5 Unit Test Description **30**

6 Appendix **33**
6.1 Symbolic Parameters 33
6.2 Usability Survey Questions 36

7 Appendix — Generative AI Use Disclosure **37**
7.1 AI Tools Used 37
7.2 How and Where Used 37
7.3 Verification of AI-Generated Content 37

8 Appendix — Reflection **38**

List of Tables

2 Lake Verification and Validation Team 4
3 GPU coverage for T.NFR.COMP.2 22
4 Traceability of system tests to functional requirements 27
5 Traceability of nonfunctional and safety tests to requirements 29
6 Test models 33
7 Symbolic parameters from the SRS 34
8 Symbolic parameters used only in testing 35

1 Symbols, Abbreviations, and Acronyms

Term	Description
CPU	Central processing unit. A device's general-purpose processor.
GGUF	The file format used by llama.cpp and ggml to store a model's weights, tokeniser, and metadata. Lake only accepts GGUF models.
GPL	GNU General Public License. The free software licence, version 3.0, that Lake is released under.
GPU	Graphics processing unit. A device's parallel processor, which does the arithmetic of inference.
Greedy decoding	Always choosing the most probable next token, so the same prompt gives the same output every time.
Release candidate	A build of Lake that is intended to become a release or be used in a demonstration. The manual tests marked to run per release candidate are run on it before it is tagged.
SOLID	Acronym for specifying design principles. Stands for Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion.

This document is the Verification and Validation (V&V) Plan for Lake. The General Information section describes what is being tested and why. The Plan section covers how we verify the SRS, the design, this plan, and the implementation, and how we validate the system with users. The System Tests section lists the tests for the functional, nonfunctional, and safety requirements and traces each test to a requirement. The Unit Test Description will be completed after the design documents. The appendix holds the symbolic parameters and the usability survey.

2 General Information

This plan covers the core release of Lake as scoped in the SRS.

2.1 Summary

The software under test is Lake, which runs one large language model (LLM) across several consumer devices on a local network. Lake splits the model by layer and gives each device a consecutive range of layers. The operator forms a Lake network from the Lake Client, loads a GGUF model, and sends prompts through an OpenAI-compatible API. When a node joins or leaves, Lake reassigns the layers and keeps serving requests.

All five components are in scope: the Coordinator, the Model Partitioner, the Inference Engine, Tensor I/O, and the Lake Client.

2.2 Objectives

The qualities we most need confidence in are output correctness, privacy, reliability when devices come and go, and performance.

Splitting a model must not change what it produces. The split model's benchmark scores must stay within *BENCH_RATIO* of the unsplit model's. Related to NFR.S.6.

No node may keep request data after a request finishes, and nothing from a request may leave the Lake network except the response to the client. Related to NFR.S.10, NFR.S.11, and SR5-2.

For reliability, Lake must keep serving requests when a node fails, leaves, or joins, and must refuse a model that would run a node out of memory. Related to FR.S.1.10 to FR.S.1.12, SR1-3, and SR2-1.

For performance, a Lake network must run a model too large for its biggest node, and the number of requests it serves at once within *MAX_DEGRADATION* must grow with the number of nodes. Single-request speed is measured but has no pass threshold, since a split model is not expected to beat one device on a model that already fits. Related to NFR.S.4 and NFR.S.5.

Usability is checked with a session with new users and the survey in the appendix. For platforms, we test the operating system, CPU, and GPU combinations the team owns and report the rest as untested in the V&V Report.

We test how Lake uses ggml and llama.cpp, not the libraries themselves. The content a correctly running model generates is also out of scope. The stretch goals (encryption, authentication, wireless links, and smartphones) are not tested this year, and neither are attacks from inside the local network.

2.3 Custom Documents

Our custom documents are a Design Thinking Report in the fall and a Performance Report in the winter (Chambachan et al., 2026e). Software validation checks the system against the user needs recorded in the Design Thinking Report. The Performance Report measures model size, token throughput, latency, and reassignment time beyond the SRS targets, starting from the results of the performance tests in this plan.

2.4 Relevant Documentation

Every system test traces to a requirement in the SRS (Chambachan et al., 2026f), which also defines the symbolic constants used here. The Hazard Analysis (Chambachan et al., 2026b) produced the safety requirements SR0-1 to SR5-2, and each has at least one test. Five of its 12 failure modes are in communication between nodes, so most of our fault-injection tests target that area.

The Development Plan (Chambachan et al., 2026a) sets the linters, test frameworks, and continuous integration pipeline our automated tests use. Unit tests will be written against the Module Guide (Chambachan et al., 2026c) and Module Interface Specification (Chambachan et al., 2026d) once they exist.

3 Plan

The following is a set of verification practices that the Lake team will use to ensure that requirements, design, implementation, and testing are each verified throughout the development process.

3.1 begins by establishing team roles specifically for project verification. Following this, 3.2, 3.3, and 3.4 specify how the team will verify various non-code artifacts - the SRS (Chambachan et al., 2026f), design, and the verification and validation plan (this document).

Finally, 3.5 - Implementation Verification, 3.6 - Automated Testing and Verification Tools, and 3.7 - Software Validation provide further specification on how code artifacts will be validated and verified.

3.1 Verification and Validation Team

The team will take on overall responsibility for the execution of Verification and Validation, based on the following roles and responsibilities.

Table 2: Lake Verification and Validation Team

Team Member	**Role**	**Verification and Validation Responsibilities**
Gary Qin	Lead	Oversight of the execution of the plan. Responsible for promoting changes brought forth by R&D, ensuring new tools are implemented, and that automated verification and pipeline tools are operating as intended.
Jinwoo Hong	Research & Development	Research of state of the art verification/validation tooling and providing recommendations to the team. If required, small-scale integration of new validation tools for demonstration purposes and to prove effectiveness.
Matthew Nesbitt	DevOps & CI-CD	Integration of identified tools and practices into the project's pipeline, ensuring that all code and documentation changes are checked automatically for adherence. Repair and fixes to CI-CD as required.
Colin Chambachan	Implementation	Working with Coordinator and Research & Development team members to implement tools and practices in the codebase. This applies for document- and code-based verification and validation.
Ethan Walsh	Coordinator	Ensuring that each component of the plan has been executed to the team's quality standard. Additionally, tracking updates to plan and updating documentation to reflect what is up-to-date.

3.2 SRS Verification

In order to ensure that the SRS meets expectations, and that the project meets what was originally set out in the SRS, the following verification methods will be used:

- **External Peer Review**: Since the team aims to validate their project externally and Lake has no designated supervisor, the team will seek external feedback from a peer who is not a member of the Capstone course. This peer should be familiar with the project and its goals at a high-level, possess relevant background knowledge, and can perform an ad-hoc review. This allows the team to surface new insights and improve the readability and quality of their SRS.

- **Internal Peer Review**: As with each document in the Capstone project, there is a review process performed by another team. This review aims to find consistency and content issues within the document, and will be used to provide the team additional feedback.

- **GitHub Issue Tracking**: All feedback provided to the team will be noted and converted into issues on GitHub. This allows the team to maintain a backlog of SRS-related issues and fix identified problems.

- **Internal Team Verification** (Once per deliverable): As the team progresses through the term, they will review their progress to ensure it aligns with what was set out in the SRS document. In the case new code or documentation does not align with the SRS, the team will discuss the necessary fix. This will result in either the SRS being amended to reflect the project more accurately, or the deliverable will be changed. The goal of this process is to guarantee consistency: what is stated in any code or documentation after the SRS should logically follow from what was prescribed in it.

- **SRS Checklist**: In order to complete internal team verification, the lead for verification and validation (as set out in 3.1) will create a checklist of criteria. This checklist will be used by the team to ensure each deliverable logically follows from the SRS, and if not, the necessary changes have been completed and tracked sufficiently.

3.3 Design Verification

Design verification of Lake will begin after November 16, when the first design document will be complete. In order to verify the design of Lake, the following methods will be used:

- **Peer Review**: Similar to verification of the SRS that was detailed in 3.2, peer review will be used to verify the design documentation. This will involve the traditional peer review process from another Capstone team, along with external review. External review will be used as a method to challenge the team's assumptions and point out potential design flaws.
- **Inspection of Module Guide and Module Interface Specification**: After the document has been completed, the team will perform a review of the design documentation. This will be used to identify inconsistencies, gaps between design and requirements, and flag areas of concern. Any areas of concern should be noted as GitHub issues and resolved prior to further design work.
- **Checklist-based Verification**: The team will combine results of the peer review along with an existing checklist to complete the design verification process. The checklist will focus on the following:
  - Adherence to general software design principles and practices (e.g., SOLID)
  - Adherence to modular design principles (minimal effort to build additional features)
  - Consideration for design scaling and performance-oriented non-functional requirements (ability to scale and expand the design to accommodate additional requests, devices, and traffic).

3.4 Verification and Validation Plan Verification

In order to verify the quality of the team's verification and validation plan, the team will employ the following methods:

- **Internal Review**: Throughout the preparation of the document, the team will review individual document sections and how they connect to guarantee quality and consistency.

- **Peer Review**: The team will use the peer review process to catch additional issues or inconsistencies. This will be used to complement the completed internal review.

- **Ad-Hoc Team Meeting**: The team will schedule a one-time meeting after the delivery of the VnV plan to assess its feasibility, as well as assign responsibilities to each member. This meeting will aim to assess the effectiveness and practicality of the current plan, and make changes if required.

3.5 Implementation Verification

The implementation will utilize a combination of code review, tests, benchmarks, and issue tracking methods to verify correctness and performance in an organized and auditable manner.

- **Code review:** GitHub pull requests will be created for code review following the Development Plan's process of approving code changes. At least two reviewers will check for correctness, error handling, and adequate tests for the code changes. A max of 400 lines added per pull request constraint is added to ensure each pull request is auditable for reviewers, with larger features split into several pull requests.

- **Unit and integration tests:** Unit tests will be used to ensure correctness in areas such as model layer allocation constraints, lifecycle transitions of node health, and message format validation to ensure compatibility with interfaces (e.g., the OpenAI API spec). Integration tests will be used to validate runtime behavior of running multiple node processes on a single machine with a small model to ensure functional end to end behavior. Both checks will run in the CI using GitHub runners, configured such that the relevant tests respective to the code changes are run.

- **Fault Handling:** Tests will introduce faults in lifecycle and data states to inject stale health data, unavailable nodes, corrupted or incomplete tensor messages, and communication delays. The results will be checked against the defined safety requirements in bounded recovery, explicit failures, and filtration of sensitive data in diagnostic logs.

- **System Testing:** Tests for Lake's end-to-end behavior that may require multiple devices, various supported GPU backends and operating systems, and varying degrees of system capabilities will be run on team hardware before major releases or demonstrations. System test coverage will identify the operating system, CPU architecture, GPU backends, and system capability metrics (RAM/VRAM, CPU/GPU performance). Combinations of system configurations and capabilities that are are not tested will be reported as untested in the V&V report. Each recorded result will also identify the commit of the current code version, the model as well its quantization, node configuration, and test inputs for repeatability.

- **Performance Verification:** Performance metrics such as Lake's capacity for model size, concurrent request throughput, and output quality will be used to verify that optimizations result in quantitative performance gain, and that changes do not regress performance, as specified in Section 4.2.

- **Defect Tracking:** Failed CI checks will block pull requests from being merged. If review findings are too large in scope to be addressed in the pull request, they will instead be filed as GitHub issues. Regression tests will be added when a failure is identified to prevent it from recurring.

3.6 Automated Testing and Verification Tools

The following testing and verification tools will be used in accordance with the technologies specified in the Development Plan.

- **Testing frameworks:** `pytest` (pytest developers) will be used for Python components, `cargo test` (Rust project developers) for Rust components, and `Vitest` (Vitest contributors) for TypeScript components. Tests will cover behavior around the components of the coordinator, model partitioner, inference engine, Tensor I/O, and the Lake client.

- **Static checks:** `Ruff` (Astral, 2026) will be used for Python code, `Clippy` (Rust project developers, 2026a) and `rustfmt` (Rust project developers, 2026b) will be used for Rust code, and `ESLint` (OpenJS

Foundation, 2026) and `Prettier` (Prettier contributors, 2026) will be used for TypeScript code. These checks will be run on commit hooks and pull request CI.

- **Continuous integration:** GitHub Actions (GitHub) will be used to run checks and automated tests on Linux, Windows, and macOS runners. To minimize CI minutes, applicable tests will use lightweight models across local processes. Tests requiring specific hardware that GitHub runners do not provide will instead be run on team hardware outside of the CI.

- **Code coverage:** `pytest-cov` (pytest-cov contributors) will be used for Python code coverage, and `cargo-llvm-cov` (cargo-llvm-cov contributors) will be used for Rust code coverage. Code coverage reports will be generated that summarize which components and code are not exercised by a test suite.

- **Memory and network checks:** `AddressSanitizer` (LLVM Project) will be used for verifying unsafe `ggml` (ggml-org contributors, 2026a) integrations. `tc netem` (iproute2 contributors) on Linux will be used for packet loss and delays as specified in Section 4.2.

- **Performance comparisons:** Scripts will be used to measure concurrent request throughput, as well as compare model accuracy scores against the same model served by `llama-server` (ggml-org contributors, 2026b) as defined in Section 4.2.

3.7 Software Validation

Software validation assesses if Lake meets the needs described by the SRS goals, stakeholder interview, and user profiles. In addition to checking these requirements, the team will gather feedback on whether the workflow and user experience are useful and intuitive.

- **Task-based user sessions:** Participants who have not used Lake, such as other students outside of the capstone team, will complete tasks to set up and utilize the Lake network as specified in Section 6.2. These sessions will utilize the participant count and time limits specified in Section 6.1. With the data recorded from these sessions, the

team will gauge whether users can correctly and easily configure Lake and understand how participating devices work, as well as determine the throughput and bottlenecks of the system.

- **Demonstrations of benefits:** During the PoC and revision 0 demonstration, the team will demonstrate the functionality of Lake at its current stage to collect feedback on its usefulness and benefits. Demonstrations will show a client that is connected to the Lake API to access a model via a pooled device network, and measurements across concurrent requests, model capacity, and output quality will be made, corresponding to the tests in Section 4. Participants of the Lake network at this stage can also comment on if the response times and token throughput of the model are acceptable for their use case.

- **Stakeholder interview:** Subject to availability of the previously interviewed AI developer from the SRS, the team will seek follow-up feedback. The follow-up will involve the ergonomics of integrating Lake into the company's workflow, the implementation of the security and telemetry requirements, and any limitations that would prevent operation at their company.

Feedback from the software validation will be recorded as GitHub issues and used to iterate on the implementation or revise requirements.

4 System Tests

This section contains system-level tests to comprehensively validate all functional and non-functional requirements from the Lake SRS, and the safety requirements from the Hazard Analysis. These tests are intended to be run once the Lake system is complete, using a variety of models and devices to simulate real-world use.

4.1 Tests for Functional Requirements

The tests below are grouped by which functional modules are primarily tested by each test. Each test lists which functional requirements from Section S.2 of the SRS they verify to ensure full coverage of all SRS S.2 functional requirements with the exception of:

- *FR.S.2.3* (Model Partitioner): Testing true equivalency requires deeper access to internal graph structures than is accessible to a system-level test.

Unless otherwise stated, the following can be assumed for all tests in subsection 4.1:

- The model loaded in testing is *SMALL_MODEL*, a GGUF file defined in Section 6.1.
- Multiple nodes are simulated using multiple processes on the same machine.
- Network connections are simulated between processes using the loopback interface.
- Network throttling/instability will be simulated using tc netem (iproute2 contributors).
- Tests outside of subsubsection 4.1.5 performing actions typically done through the Lake Client, are performed by sending requests through the REST API used by the Lake Client (see SRS Section S.3). This allows said tests to be automated by avoiding interaction with the UI.

4.1.1 Coordinator (S.2.1)

1. T.FR.CRD.1
   Control: Automatic
   Initial State: Lake loaded on *MIN_NODES* devices.
   Input: On the leader device, invite the other device to be part of a cluster. On each other device, accept the request.
   Output: Both devices report being in the network, and report being ready to load a model.
   Derivation: Verifies *FR.S.1.4*, *FR.S.1.5*, *FR.S.1.15*, *FR.S.1.17*, *FR.S.1.18*, *FR.S.1.20*, *FR.S.1.21*, *FR.S.5.1*, *FR.S.5.2*, *FR.S.5.7*, *FR.S.1.16*, *FR.S.4.1*.

2. T.FR.CRD.2
   Control: Automatic
   Initial State: Lake network created with *MIN_NODES* devices.

Input: Load GGUF file into network.
Output: All devices report successful loading, and report ready for inference.
Derivation: Verifies *FR.S.1.1*, *FR.S.1.6*, *FR.S.1.8*, *FR.S.1.19*, *FR.S.3.1*.

3. T.FR.CRD.3
Control: Automatic
Initial State: Model loaded and serving requests in Lake network with *MIN_NODES* devices.
Input: Reduce the throughput of a follower node below *FAIL_THRESHOLD*.
Output: The Lake network rebuilds and redistributes the model across the nodes, excluding the modified node. Any in-progress requests are retried, and the network begins accepting new requests.
Derivation: Verifies *FR.S.1.7*, *FR.S.1.10*, *FR.S.1.11*, *FR.S.1.13*, *FR.S.1.22*, *FR.S.1.23*, *FR.S.1.2*, *FR.S.1.3*, *FR.S.3.4*, *FR.S.4.5*.

4. T.FR.CRD.4
Control: Automatic
Initial State: Model loaded and serving requests in Lake network with *MIN_NODES* devices.
Input: Add a new device to the Lake network.
Output: The Lake network rebuilds and redistributes the model across the nodes, including the newly added node. Any in-progress requests are retried, and the network begins accepting new requests.
Derivation: Verifies *FR.S.1.12*, *FR.S.1.13*.

5. T.FR.CRD.5
Control: Automatic
Initial State: Model loaded in Lake network with *MIN_NODES* devices.
Input: Leader node is killed.
Output: After *LEADER_TIMEOUT*, the Lake network disbands and each follower node unloads its model partition.
Derivation: Verifies *FR.S.1.24*, *FR.S.1.25*, *SR4-2*.

4.1.2 Model Partitioner (S.2.2)

1. T.FR.MP.1
Control: Automatic

Initial State: Lake network created with *MIN_NODES* devices.
Input: Load GGUF file into network.
Output: Each device i reports Lake's GPU-accessible memory usage of not more than $M_i(1 - \mathit{MEMORY_MARGIN})$, where M_i is the device's free GPU-accessible memory before loading. The ratio of the usage of GPU-accessible memory between devices is proportional to device capabilities.
Derivation: Verifies *FR.S.2.1*, *FR.S.2.4*, *FR.S.1.8*.

2. T.FR.MP.2
Control: Automatic
Initial State: Lake network created with *MIN_NODES* devices.
Input: Load a GGUF file where $S_{\text{model}} + S_{\text{KV}} > (1 - \mathit{MEMORY_MARGIN}) \sum_i M_i$, where S_{model} is the size of the model weights, S_{KV} the size of the KV cache for *CONTEXT_MAX* tokens, and M_i the free GPU-accessible memory of device i.
Output: Leader node reports that there is no valid partition across the network.
Derivation: Verifies *FR.S.2.2*.

4.1.3 Inference Engine (S.2.3)

1. T.FR.IE.1
Control: Automatic
Initial State: Lake network created and model file loaded with *MIN_NODES* devices.
Input: Send inference request to OpenAI compatible API.
Output: Network outputs model response via OpenAI compatible API.
Derivation: Verifies *FR.S.3.2*, *FR.S.3.3*, *FR.S.4.3*, *FR.S.4.4*, *FR.S.1.9*.

2. T.FR.IE.2
Control: Automatic
Initial State: Lake network created and model file loaded with *MIN_NODES* devices.
Input: Send concurrent inference requests to OpenAI compatible API.
Output: Network outputs model responses concurrently via OpenAI compatible API.
Derivation: Verifies *FR.S.3.5*, *FR.S.4.6*, *FR.S.1.14*.

4.1.4 Tensor I/O (S.2.4)

1. T.FR.TIO.1
   Control: Manual
   Initial State: Lake network created and serving requests with *MIN_NODES* devices.
   Input: Reduce the throughput of a connection between nodes by *THROUGHPUT_REDUCTION*.
   Output: Token throughput initially drops, then CPU usage increases on both the sending and receiving devices due to increasing compression. Throughput then increases.
   Derivation: Verifies *FR.S.4.2*.

4.1.5 Lake Client (S.2.5)

1. T.FR.LCLI.1
   Control: Manual
   Initial State: Lake running on *MIN_NODES* devices on the same network.
   Input: User scans for devices, selects the desired devices, and creates a network with them. The user then accepts the join requests on each device's Lake Client.
   Output: All devices report successfully forming a network.
   Derivation: Verifies *FR.S.5.1*, *FR.S.5.2*, *FR.S.5.3*, *FR.S.5.7*.

2. T.FR.LCLI.2
   Control: Manual
   Initial State: Lake running and model loaded on *MIN_NODES* devices on the same network.
   Input: Inference request is issued through the OpenAI compatible API.
   Output: Performance metrics for the network and each node are displayed by the Lake Client. The weakest links in the system forming bottlenecks are highlighted by the Lake Client.
   Derivation: Verifies *FR.S.5.5*, *FR.S.5.6*.

3. T.FR.LCLI.3
   Control: Manual
   Initial State: Lake network created with *MIN_NODES* devices.

Input: Load a GGUF file where $S_{\text{model}}+S_{\text{KV}} > (1-\mathit{MEMORY_MARGIN})\sum_i M_i$, where S_{model} is the size of the model weights, S_{KV} the size of the KV cache for *CONTEXT_MAX* tokens, and M_i the free GPU-accessible memory of device i.
Output: Lake Client reports that there is no valid partition across the network.
Derivation: Verifies *FR.S.5.4*.

4.2 Tests for Nonfunctional Requirements

This section gives the system tests for the nonfunctional requirements in SRS S.2 (NFR.S.1 to NFR.S.15) and the Environmental Requirements (NFR.E.1 to NFR.E.8). The tests follow the verification groups in SRS S.8, and they are grouped by the same categories the SRS uses. The safety requirements are tested in Section 4.3.

Each test uses one of the three test models in Table 6. Unless a test says otherwise, generation uses greedy decoding, the *TEST_PROMPTS* fixed prompts in the test repository, and *GEN_TOKENS* generated tokens per prompt. Named values in italics are defined in Section 6.1.

4.2.1 Usability

These tests cover NFR.S.1 (understandability), NFR.S.2 (controllability) and NFR.S.3 (installability). The first is a session with people who have not used Lake, which is the only way to check how a new user experiences the interface. Its tasks and questions are in Section 6.2.

1. T.NFR.USE.1
   Type: Dynamic, manual

   Initial State: Three devices run Lake on one local network. One device has a background load that cuts its token throughput to about half, so it is the bottleneck. *DEMO_MODEL* is on the device the participant uses.

   Input/Condition: At least *TEST_USERS* participants who have not used Lake, such as students from other capstone teams, each complete the two tasks in Section 6.2.

   Output/Result: Every participant completes task 1 (create a Lake network) and task 2 (name the participating devices, the token

throughput and the bottleneck) within *UI_TASK_TIME* each, and the throttled device is highlighted as the bottleneck (FR.S.5.6). The answers to the follow-up questions are recorded and each problem raised by more than one participant becomes a GitHub issue.

How test will be performed: One team member runs the session and reads the tasks aloud without giving hints. A second team member times each task from the moment the participant first sees the interface and records whether it was completed. The participant then answers the questions in Section 6.2.

2. T.NFR.USE.2
   Type: Dynamic, manual

   Initial State: A clean machine for each operating system with no previous Lake installation, and the release installer or package for that system.

   Input/Condition: A team member installs Lake following only the installation instructions.

   Output/Result: On Windows and macOS, installation takes at most *INSTALL_CLICKS* clicks after the installer is downloaded. On Linux it takes one package manager command. On every system, Lake starts after installation without further setup.

   How test will be performed: The tester counts every click from opening the installer to the first launch of Lake, and records the count and any manual step needed. The test is repeated for each release candidate.

4.2.2 Performance

These tests cover NFR.S.4 (model size), NFR.S.5 (scalability) and NFR.S.6 (intelligence). T.NFR.PERF.2 measures how throughput changes with load, so part of its output is a table and a graph rather than a single pass or fail.

1. T.NFR.PERF.1
   Type: Dynamic, manual

   Initial State: A Lake network using devices owned by the team that cannot individually hold *CAPACITY_MODEL*: two Windows laptops

with NVIDIA GPUs, a MacBook, and a Linux PC with an AMD GPU. Let $M_{\max}$ be the largest free GPU-accessible memory of any node and M_{total} the sum over all nodes, each reduced by *MEMORY_MARGIN*. *CAPACITY_MODEL* is used at the lowest precision whose memory need (weights plus the KV cache for *CONTEXT_MAX* tokens) is greater than $M_{\max}$ and at most M_{total}.

Input/Condition: First, the model is loaded on the node with the most memory alone. Then it is loaded across the whole Lake network and given the test prompts.

Output/Result: Loading on one node fails with a partitioning failure (FR.S.2.2). Loading across the network succeeds, and the responses meet the criteria of the inference correctness tests in Section 4.1.

How test will be performed: The memory of each node is read from the Coordinator's device report, and the precision is chosen from those values and recorded with the result. With the devices above, $M_{\max}$ is about 14.4 GB (the AMD GPU), so the rule selects 16-bit precision, which needs about 15 GB. The test is run once per demonstration with the devices available at the time.

2. T.NFR.PERF.2
   Type: Dynamic, automated

   Initial State: Lake networks of $k = 1, 2, \ldots$ nodes, up to the number of team devices available, running *DEMO_MODEL*.

   Input/Condition: For each k, the number of concurrent requests c is increased through $1, 2, 4, 8, \ldots$ while each request generates *GEN_TOKENS* tokens.

   Output/Result: For each k, the largest c, called $c_{\max}(k)$, at which the mean per-request token throughput is within *MAX_DEGRADATION* of the single-request throughput. The test passes if $c_{\max}(k) \geq$ *MIN_CONCURRENT* for every $k \geq 2$ and $c_{\max}(k) \geq k \cdot c_{\max}(1)$. The results are also reported as a table and a graph of $c_{\max}$ against k.

   How test will be performed: A load-test script sends the requests through the OpenAI-compatible API and records the tokens per second of each request. The results are kept for the Performance Report.

3. T.NFR.PERF.3
   Type: Dynamic, automated

   Initial State: *DEMO_MODEL* served two ways: by llama.cpp's `llama-server` on one machine, and by a Lake network of *MIN_NODES* nodes.

   Input/Condition: The same *BENCH_QUESTIONS* multiple-choice questions from the MMLU (Massive Multitask Language Understanding) benchmark (Hendrycks et al., 2021), a standard test of a language model's knowledge and reasoning with four-option questions across 57 subjects, such as mathematics, law and medicine. The questions are sent to both through the OpenAI-compatible API with greedy decoding.

   Output/Result: The Lake network's accuracy is at least *BENCH_RATIO* of the single-machine accuracy. The relative error $\frac{|a_{\text{single}} - a_{\text{Lake}}|}{a_{\text{single}}}$ is reported, where a is the fraction of questions answered correctly.

   How test will be performed: A script sends each question, extracts the answer letter from the response, and scores both runs against the answer key. Using `llama-server` as the baseline means both runs use the same API and prompts, so only the split differs.

4.2.3 Look and Feel

NFR.S.7 (consistency) is checked by inspection, since it concerns how the Lake Client's source is written rather than how it behaves.

1. T.NFR.LF.1
   Type: Static, inspection

   Initial State: The Lake Client source at a release candidate.

   Input/Condition: Two team members who did not write the Lake Client code under review go through this checklist:

   (a) A search of the source for colour values (hexadecimal, `rgb()` and `hsl()`) and font names finds them only in the theme file.

   (b) Every component takes its colours and fonts from the theme.

(c) Every screen of the Lake Client, viewed once, shows no colour or font outside the theme.

Output/Result: All three items pass. Any failure is filed as a GitHub issue.

How test will be performed: The reviewers record the result of each item and the commit reviewed in the pull request for the release candidate.

4.2.4 Fault Tolerance

These tests cover NFR.S.8 (rejecting invalid model files) and NFR.S.9 (correct results despite packet loss and corruption).

1. T.NFR.FT.1
   Type: Dynamic, automated

   Initial State: A Lake network of *MIN_NODES* nodes on one machine, with no model loaded.

   Input/Condition: Four invalid model files, each given to the Lake Client as the model to load:

   (a) *SMALL_MODEL* truncated to half its size;

   (b) *SMALL_MODEL* with its first four bytes (the GGUF magic number) overwritten;

   (c) a plain text file renamed with a `.gguf` extension;

   (d) a valid GGUF file of a model architecture Lake does not support.

   Output/Result: Each file is rejected with an error message that names the problem, no partition is sent to any node, and no process crashes.

   How test will be performed: A test script loads each file in turn and checks the error returned, the nodes' logs, and that every process is still running.

2. T.NFR.FT.2
   Type: Dynamic, automated

   Initial State: A Lake network of *MIN_NODES* nodes on one Linux machine running *SMALL_MODEL*, with the nodes connected through the loopback interface.

Input/Condition: The test prompts run once with no network impairment and then again for each packet loss and corruption rate in *LOSS_LEVELS*.

Output/Result: The generated tokens under every impairment are identical to the tokens with no impairment.

How test will be performed: The Linux `tc netem` tool adds the loss or corruption to the loopback interface. The test runs in continuous integration on a Linux runner.

4.2.5 Privacy

These tests cover NFR.S.10 (no stored request data), NFR.S.11 (no data sent outside the Lake network), NFR.S.12 (no untrusted devices) and NFR.E.5 (no requests to third-party services).

1. T.NFR.PRIV.1
   Type: Dynamic, automated

   Initial State: A Lake network of *MIN_NODES* nodes with *SMALL_MODEL* loaded and logging at its most detailed level.

   Input/Condition: The test prompts, each with a unique random marker string added, sent through the API until every request completes. The prompts are also sent during the faults of T.SR.3 and T.SR.4, since error handling is where request data is most likely to reach a log.

   Output/Result: A search of every file Lake wrote on every node, including its data directory, diagnostic logs and temporary files, finds none of the marker strings, none of the response text, and no tensor or KV cache contents (SR5-2).

   How test will be performed: A script records the files Lake creates or changes during the run and searches them for each marker and response. Tensor contents are found by searching for the byte patterns of tensors sent during the run.

2. T.NFR.PRIV.2
   Type: Dynamic, manual

   Initial State: A Lake network of *MIN_NODES* nodes on a local network, with packet capture running on every node.

Input/Condition: Leave an open session for *IDLE_TIME* with no requests, then run the test prompts, and shut down the Lake network.

Output/Result: Every captured packet sent by Lake goes to another node of the Lake network, to a device on the local network being discovered or invited, or to the client that sent the request. There are no other destinations, including no name lookups for outside hosts.

How test will be performed: A team member captures traffic with `tcpdump` on Linux and macOS or Wireshark on Windows, filters it to the Lake processes, and lists every destination address and port.

3. T.NFR.PRIV.3
   Type: Dynamic, manual

   Initial State: The Lake network formation setup from Section 4.1: one leader and three other devices running Lake on one local network, with packet capture running on each.

   Input/Condition: The three devices cover each case of the two approvals in SR5-1, where the leader approves a device by selecting it (FR.S.5.2). The leader selects two of the devices and invites them: one accepts the invitation and one declines it. The third device is not selected, so it is never invited. The leader then loads *SMALL_MODEL* and runs the test prompts.

   Output/Result: Only the leader and the device that both the leader and the device approved take part in the Lake network. The other two devices receive no model partition, hidden state or request data, and each of them appears in the capture only in discovery and invitation traffic.

   How test will be performed: A team member filters the capture on each of the two excluded devices to the Lake processes and checks every packet received against this rule.

4.2.6 Compatibility

These tests cover NFR.S.13 (operating systems), NFR.S.14 (CPU architectures), NFR.S.15 (GPUs) and NFR.E.1 (supported platforms). Operating systems and CPU architectures are covered by continuous integration. Continuous integration runners have no usable GPUs, so each GPU vendor is covered on team-owned devices, as shown in Table 3.

Table 3: GPU coverage for T.NFR.COMP.2

Vendor	**Verification**
NVIDIA	Run with the CUDA backend on Gary's laptop (GeForce RTX 3050 Laptop GPU, 4 GB) and Colin's Windows laptop (8 GB).
Apple	Run with the Metal backend on Jinwoo's MacBook (64 GB unified memory), Ethan's MacBook (32 GB) and Colin's MacBook (16 GB).
AMD	Run on Matthew's AMD GPU (16 GB) with the ROCm backend, or the Vulkan backend if ROCm does not support the card or its operating system.

1. T.NFR.COMP.1
   Type: Dynamic, automated

   Initial State: The continuous integration workflow, with runners for Windows, macOS and Linux on both x86_64 and ARM where GitHub provides them.

   Input/Condition: Every pull request to the main branch.

   Output/Result: Lake builds, and its unit and integration tests pass with the CPU backend, on every runner.

   How test will be performed: A GitHub Actions matrix job runs the build and the tests on each runner. The branch rules block merging if any runner fails.

2. T.NFR.COMP.2
   Type: Dynamic, manual

   Initial State: The devices in Table 3, each running Lake with its GPU backend.

   Input/Condition: The inference correctness test from Section 4.1 with *DEMO_MODEL*, split across two processes on the device.

   Output/Result: On each device, the split model produces the same tokens as the unsplit model on the same device.

   How test will be performed: The device owner runs the test before each demonstration and records the device, backend and result.

4.2.7 Environment and Licensing

These tests cover NFR.E.4 (model weights unchanged), NFR.E.6 (formatting and linting), NFR.E.7 (GPL compliance) and NFR.E.8 (model licences).

1. T.NFR.ENV.1
   Type: Dynamic, automated

   Initial State: *SMALL_MODEL* on disk, with its SHA-256 checksum recorded.

   Input/Condition: A Lake network loads the model, partitions it across *MIN_NODES* nodes and runs the test prompts.

   Output/Result: The model file's SHA-256 checksum is unchanged.

   How test will be performed: Part of the integration tests run in continuous integration.

2. T.NFR.ENV.2
   Type: Static, automated

   Initial State: The continuous integration workflow.

   Input/Condition: Every pull request to the main branch.

   Output/Result: Ruff (Astral, 2026) and Black (Black contributors, 2026) pass for Python, rustfmt (Rust project developers, 2026b) and Clippy (Rust project developers, 2026a) for Rust, and ESLint (OpenJS Foundation, 2026) and Prettier (Prettier contributors, 2026) for TypeScript, with no errors.

   How test will be performed: The linting stage of continuous integration runs each tool. The branch rules block merging if it fails.

3. T.NFR.ENV.3
   Type: Static, automated

   Initial State: The project's dependency lists for Rust, Python and TypeScript.

   Input/Condition: Every pull request that changes a dependency.

   Output/Result: Every dependency's licence is on the project's list of licences compatible with GPL v3.0 (Free Software Foundation, 2007).

How test will be performed: `cargo-deny` (Embark Studios, 2026), `pip-licenses` (raimon49, 2026) and `license-checker` (Glass, 2026) run in continuous integration and fail the build for any licence not on the list.

4. T.NFR.ENV.4
   Type: Static, inspection

   Initial State: The list of models used in testing and demonstrations (Table 6).

   Input/Condition: Two team members read the licence of each model.

   Output/Result: Each model's licence permits the way Lake uses it, and the licence name and link are recorded in the test repository.

   How test will be performed: The review is repeated whenever a model is added to the list.

4.3 Tests for Safety Requirements

This section gives the system tests for the safety requirements from the Hazard Analysis (SR0-1 to SR5-2). SR5-1 (two-sided authorisation) and SR5-2 (no request data in diagnostic logs) are the same properties as NFR.S.12 and NFR.S.10, so they are covered by T.NFR.PRIV.3 and T.NFR.PRIV.1. Several tests need faults that cannot be caused from outside Lake, such as a corrupted tensor that reaches Tensor I/O, since TCP, which gRPC runs on, discards most data corrupted on the network. For these, test builds of Lake include a fault-injection option in Tensor I/O and the Coordinator. It is not compiled into release builds.

1. T.SR.1
   Type: Dynamic, automated

   Initial State: The Model Partitioner, given device reports for test Lake networks of one to four nodes with different amounts of free memory.

   Input/Condition: *SMALL_MODEL* and *DEMO_MODEL*, each partitioned for every test network, together with three invalid cases: a model larger than the whole network's memory after *MEMORY_MARGIN* is removed from each node, a manual allocation

that puts more layers on one node than its memory allows, and a network of one node.

Output/Result: Every partitioning produced uses at least *MIN_NODES* nodes, assigns every layer of the model to exactly one node, and gives every selected node at least one layer. Each invalid case is rejected with a partitioning failure (FR.S.2.2) and nothing is sent to any node.

How test will be performed: Part of the integration tests run in continuous integration. The test checks each partitioning against the rules above and the memory reported for each node.

2. T.SR.2
   Type: Dynamic, automated

   Initial State: A three-node Lake network on one machine running *SMALL_MODEL*.

   Input/Condition: The test prompts are sent while each of the following happens in turn:

   (a) one follower's health reports are held back for longer than *HEALTH_TTL*;

   (b) one follower is made to fail its health check, then made to pass it again;

   (c) a fourth node joins and its partition is not yet confirmed.

   Leader failure, the other case of SR4-2, is tested by T.FR.CRD.5.

   Output/Result: In cases (a) and (b) the follower is marked unavailable and is not given new requests or partitions until it passes a health check. In case (c) no request is sent to the new node before its partition is confirmed.

   How test will be performed: Part of the integration tests run in continuous integration. The fault-injection option holds back health reports and fails health checks. The Coordinator's routing log records which node each request is sent to.

3. T.SR.3
   Type: Dynamic, automated

Initial State: A Lake network of *MIN_NODES* nodes on one machine running *SMALL_MODEL*.

Input/Condition: During a request, the follower process is paused. In one run it is resumed before *RECOVERY_LIMIT* attempts are used. In a second run it stays paused.

Output/Result: In both runs the Coordinator treats the connection as failed after *NODE_TIMEOUT* without a response. In the first run the request is retried and completes with the same tokens as a run with no fault. In the second run the request is retried at most *RECOVERY_LIMIT* times and the API then returns an error for that request instead of waiting forever.

How test will be performed: Part of the integration tests run in continuous integration. The script pauses and resumes the follower with operating system signals and records the time of each retry from the Coordinator's log.

4. T.SR.4
   Type: Dynamic, automated

   Initial State: A Lake network of *MIN_NODES* nodes on one machine running *SMALL_MODEL*.

   Input/Condition: The fault-injection option changes bytes in one tensor message and cuts another tensor message short, during different requests.

   Output/Result: Each damaged tensor is detected and discarded, a failure is reported to the Coordinator, and the request is retried and completes with the same tokens as a run with no fault. The log entry for each failure holds the tensor's metadata, such as its size and the request and layer it belongs to, and none of its contents.

   How test will be performed: Part of the integration tests run in continuous integration. T.NFR.FT.2 cannot cover this, because TCP, which gRPC runs on, discards most packets that `tc netem` corrupts before Lake reads them.

5. T.SR.5
   Type: Dynamic, automated

Initial State: A Lake network of *MIN_NODES* nodes on one Linux machine running *SMALL_MODEL*, with the nodes connected through the loopback interface and a baseline latency recorded for the connection.

Input/Condition: The test prompts, run with `tc netem` adding a delay of half of *LATENCY_TOL* above the baseline, then twice *LATENCY_TOL* above it.

Output/Result: The Coordinator is notified of high latency during the second run and not during the first.

How test will be performed: Part of the integration tests run in continuous integration on a Linux runner. The test checks the Coordinator's log for the latency notice.

4.4 Traceability Between Test Cases and Requirements

Table 4 lists the functional requirements that each system test in Section 4.1 verifies. Every functional requirement is supported by at least one system test, with the exceptions listed at the start of Section 4.1.

Table 5 lists the nonfunctional and safety requirements that each system test in Sections 4.2 and 4.3 verifies. Every nonfunctional and safety requirement appears at least once. NFR.E.2 and NFR.E.3 are also verified by the partitioning and FR.S.1.9 tests in Section 4.1. Some of these tests also check a functional requirement, such as FR.S.5.6 in T.NFR.USE.1 and FR.S.2.2 in T.NFR.PERF.1.

Table 4: Traceability of system tests to functional requirements

Test Case ID	**Requirements Verified**
T.FR.CRD.1	*FR.S.1.4*, *FR.S.1.5*, *FR.S.1.15*, *FR.S.1.17*, *FR.S.1.18*, *FR.S.1.20*, *FR.S.1.21*, *FR.S.5.1*, *FR.S.5.2*, *FR.S.5.7*, *FR.S.1.16*, *FR.S.4.1*
T.FR.CRD.2	*FR.S.1.1*, *FR.S.1.6*, *FR.S.1.8*, *FR.S.1.19*, *FR.S.3.1*
T.FR.CRD.3	*FR.S.1.7*, *FR.S.1.10*, *FR.S.1.11*, *FR.S.1.13*, *FR.S.1.22*, *FR.S.1.23*, *FR.S.1.2*, *FR.S.1.3*, *FR.S.3.4*, *FR.S.4.5*
T.FR.CRD.4	*FR.S.1.12*, *FR.S.1.13*
T.FR.CRD.5	*FR.S.1.24*, *FR.S.1.25*, *SR4-2*
T.FR.MP.1	*FR.S.2.1*, *FR.S.2.4*, *FR.S.1.8*
T.FR.MP.2	*FR.S.2.2*
T.FR.IE.1	*FR.S.3.2*, *FR.S.3.3*, *FR.S.4.3*, *FR.S.4.4*, *FR.S.1.9*
T.FR.IE.2	*FR.S.3.5*, *FR.S.4.6*, *FR.S.1.14*
T.FR.TIO.1	*FR.S.4.2*
T.FR.LCLI.1	*FR.S.5.1*, *FR.S.5.2*, *FR.S.5.3*, *FR.S.5.7*
T.FR.LCLI.2	*FR.S.5.5*, *FR.S.5.6*
T.FR.LCLI.3	*FR.S.5.4*

Table 5: Traceability of nonfunctional and safety tests to requirements

Test case	**Nonfunctional requirements**	**Safety requirements**
T.NFR.USE.1	NFR.S.1, NFR.S.2	
T.NFR.USE.2	NFR.S.3, NFR.S.13, NFR.E.1	
T.NFR.PERF.1	NFR.S.4, NFR.E.2	SR1-3
T.NFR.PERF.2	NFR.S.5, NFR.E.3	
T.NFR.PERF.3	NFR.S.6, NFR.E.3	
T.NFR.LF.1	NFR.S.7	
T.NFR.FT.1	NFR.S.8	
T.NFR.FT.2	NFR.S.9	SR3-1
T.NFR.PRIV.1	NFR.S.10	SR5-2
T.NFR.PRIV.2	NFR.S.11, NFR.E.5	
T.NFR.PRIV.3	NFR.S.12	SR5-1
T.NFR.COMP.1	NFR.S.13, NFR.S.14, NFR.E.1	
T.NFR.COMP.2	NFR.S.15	
T.NFR.ENV.1	NFR.E.4	
T.NFR.ENV.2	NFR.E.6	
T.NFR.ENV.3	NFR.E.7	
T.NFR.ENV.4	NFR.E.8	
T.SR.1	NFR.E.2	SR1-1, SR1-2, SR1-3
T.SR.2		SR0-1, SR0-2, SR4-2
T.SR.3		SR2-1, SR4-1
T.SR.4		SR3-1, SR3-2, SR5-2
T.SR.5		SR4-3

5 Unit Test Description

This section will be completed after the Module Interface Specification (Chambachan et al., 2026d), since unit tests are written against each module's interface. The areas we already plan to unit test are listed in Section 3.5.

References

Astral. Ruff: An extremely fast Python linter and code formatter. https://docs.astral.sh/ruff, 2026. Accessed 19 October 2026.

Black contributors. Black: The uncompromising Python code formatter. https://black.readthedocs.io, 2026. Accessed 19 October 2026.

cargo-llvm-cov contributors. cargo-llvm-cov. https://github.com/taiki-e/cargo-llvm-cov. Accessed 2 November 2026.

Colin Chambachan, Jinwoo Hong, Matthew Nesbitt, Gary Qin, and Ethan Walsh. Development plan for Lake. https://github.com/capstoneCEGJM/Lake/blob/main/pdfs/DevelopmentPlan/DevelopmentPlan.pdf, 2026a.

Colin Chambachan, Jinwoo Hong, Matthew Nesbitt, Gary Qin, and Ethan Walsh. Hazard analysis for Lake. https://github.com/capstoneCEGJM/Lake/blob/main/pdfs/HazardAnalysis/HazardAnalysis.pdf, 2026b.

Colin Chambachan, Jinwoo Hong, Matthew Nesbitt, Gary Qin, and Ethan Walsh. Module guide for Lake. https://github.com/capstoneCEGJM/Lake/blob/main/pdfs/Design/SoftArchitecture/MG.pdf, 2026c.

Colin Chambachan, Jinwoo Hong, Matthew Nesbitt, Gary Qin, and Ethan Walsh. Module interface specification for Lake. https://github.com/capstoneCEGJM/Lake/blob/main/pdfs/Design/SoftDetailedDes/MIS.pdf, 2026d.

Colin Chambachan, Jinwoo Hong, Matthew Nesbitt, Gary Qin, and Ethan Walsh. Problem statement and goals for Lake. https://github.com/capstoneCEGJM/Lake/blob/main/pdfs/ProblemStatementAndGoals/ProblemStatement.pdf, 2026e.

Colin Chambachan, Jinwoo Hong, Matthew Nesbitt, Gary Qin, and Ethan Walsh. Software requirements specification for Lake. https://github.com/capstoneCEGJM/Lake/blob/main/pdfs/SRS/SRS.pdf, 2026f.

Embark Studios. cargo-deny: Cargo plugin for linting your dependencies. https://github.com/EmbarkStudios/cargo-deny, 2026. Accessed 2 November 2026.

Free Software Foundation. GNU General Public License, version 3. https://www.gnu.org/licenses/gpl-3.0.html, 2007. Accessed 19 October 2026.

ggml-org contributors. ggml: Tensor library for machine learning. https://github.com/ggml-org/ggml, 2026a. Accessed 16 October 2026.

ggml-org contributors. llama.cpp: LLM inference in C/C++. https://github.com/ggml-org/llama.cpp, 2026b. Accessed 16 October 2026.

GitHub. GitHub Actions documentation. https://docs.github.com/en/actions. Accessed 2 November 2026.

Dav Glass. license-checker: Check NPM package licenses. https://github.com/davglass/license-checker, 2026. Accessed 2 November 2026.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021. https://github.com/hendrycks/test.

iproute2 contributors. tc-netem: Network Emulator manual. https://man7.org/linux/man-pages/man8/tc-netem.8.html. Accessed 2 November 2026.

LLVM Project. AddressSanitizer documentation. https://clang.llvm.org/docs/AddressSanitizer.html. Accessed 2 November 2026.

OpenJS Foundation. ESLint: Find and fix problems in your JavaScript code. https://eslint.org, 2026. Accessed 19 October 2026.

Prettier contributors. Prettier: An opinionated code formatter. https://prettier.io, 2026. Accessed 19 October 2026.

pytest-cov contributors. pytest-cov documentation. https://pytest-cov.readthedocs.io/en/latest/. Accessed 2 November 2026.

pytest developers. pytest documentation. https://docs.pytest.org/en/stable/. Accessed 2 November 2026.

raimon49. pip-licenses: Dump the software license list of Python packages installed with pip. https://github.com/raimon49/pip-licenses, 2026. Accessed 2 November 2026.

Rust project developers. The Cargo Book: cargo test. https://doc.rust-lang.org/cargo/commands/cargo-test.html. Accessed 2 November 2026.

Rust project developers. Clippy: A collection of lints to catch common mistakes in Rust code. https://doc.rust-lang.org/clippy, 2026a. Accessed 19 October 2026.

Rust project developers. rustfmt: Format Rust code. https://github.com/rust-lang/rustfmt, 2026b. Accessed 19 October 2026.

Vitest contributors. Vitest: Getting Started. https://vitest.dev/guide/. Accessed 2 November 2026.

6 Appendix

This appendix gives the symbolic parameters used by the system tests and the questions for the usability session.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

The test models are listed in Table 6. All three use the Llama architecture, so the same code path is tested from the smallest model to the largest, and all three are released under the Apache 2.0 licence. Values taken from the SRS Symbolic Constants appendix keep the value given there and are listed in Table 7. Values only used in testing are listed in Table 8.

Table 6: Test models

Name	Model and use	Size
SMALL_MODEL	SmolLM2-135M-Instruct, Q8_0. Continuous integration and tests where answer quality does not matter.	about 150 MB
DEMO_MODEL	SmolLM2-1.7B-Instruct, Q8_0. Quality comparisons, compatibility runs and the usability session.	about 1.8 GB
CAPACITY_MODEL	Mistral-7B-Instruct-v0.3, at the precision chosen in test T.NFR.PERF.1.	7.7 to 14.5 GB

Table 7: Symbolic parameters from the SRS

Name	Value	Description
UI_TASK_TIME	2 min	Time a new user may take for each usability task.
INSTALL_CLICKS	10	Maximum clicks to install on Windows or macOS.
TEST_USERS	3	Minimum number of usability session participants.
MAX_DEGRADATION	20%	Maximum loss of per-request throughput under concurrent load.
MIN_CONCURRENT	2	Minimum number of requests served at the same time.
BENCH_RATIO	95%	Minimum benchmark accuracy of the split model, relative to one machine.
MAX_LOSS	10%	Highest packet loss and corruption rate tested.
CONTEXT_MAX	4096 tokens	Context length each node reserves KV cache for.
MEMORY_MARGIN	10%	Share of each node's memory left free for runtime overhead.
MIN_NODES	2	Minimum nodes a model is partitioned across; also the default size of a test Lake network.
FAIL_THRESHOLD	50%	Share of usual throughput below which a node is failing.
HEALTH_TTL	15 s	Age after which health data is stale.
NODE_TIMEOUT	10 s	Time without a response before a connection is failed.
RECOVERY_LIMIT	3	Maximum recovery attempts for one request.
LEADER_TIMEOUT	30 s	Time without a health query before followers assume the leader failed.
LATENCY_TOL	50%	Increase over baseline latency that is reported to the Coordinator.

Table 8: Symbolic parameters used only in testing

Name	**Value**	**Description**
TEST_PROMPTS	20	Number of fixed prompts used by default.
GEN_TOKENS	128	Tokens generated per prompt.
BENCH_QUESTIONS	500	MMLU questions in T.NFR.PERF.3.
LOSS_LEVELS	1%, 5%, *MAX_LOSS*	Packet loss and corruption rates in T.NFR.FT.2.
IDLE_TIME	10 min	Idle period captured in T.NFR.PRIV.2.
THROUGHPUT_REDUCTION	50%	Network connection throughput reduction in T.FR.TIO.1.

6.2 Usability Survey Questions

This survey is used in test T.NFR.USE.1. It checks NFR.S.1 and NFR.S.2, and the bottleneck highlighting in FR.S.5.6. Each participant has a time limit of *UI_TASK_TIME* for each task. A team member records the time taken and whether the task was completed, and gives no hints during the tasks.

Tasks

1. Create a Lake network using the devices listed on the card and the model provided. (NFR.S.2)
2. Using the interface, name the devices taking part, the current token throughput, and the slowest device or connection. (NFR.S.1, FR.S.5.6)

Questions after the tasks

1. Was anything confusing while creating the network?
2. How did you find the slowest device or connection? Was anything hard to find?
3. Did you expect to see or do anything that was not there?

7 Appendix — Generative AI Use Disclosure

7.1 AI Tools Used

- Anthropic Claude Opus 5.5, used through Claude Code.
- OpenAI ChatGPT.

7.2 How and Where Used

- **4.2 Tests for Nonfunctional Requirements and 4.3 Tests for Safety Requirements**: Claude drafted the tests from the SRS and Hazard Analysis requirements and the test approach, test IDs and hardware we chose. We reviewed and edited the draft. Claude also drafted the nonfunctional and safety rows of the traceability table in 4.4, the symbolic parameters tables in 6.1 and the usability survey in 6.2.
- **2 General Information**: Claude drafted the section from the SRS, Hazard Analysis and Problem Statement, along with the bibliography entries for our documents. We reviewed and edited the draft. Claude also drafted the introduction, the Unit Test Description note and the revision history in a final pass.
- **3.5 Implementation Verification to 3.7 Software Validation**: ChatGPT was used to brainstorm and research ideas and tools alongside course material. It was also used to add placeholder LaTeX code and for a final grammar check, in these sections and in the appendix.

7.3 Verification of AI-Generated Content

Every requirement ID and symbolic constant in the drafted sections was checked against the SRS, and every requirement in 4.2 was checked to have a test in the traceability table.

The requirement IDs, failure mode counts and document references in Section 2 were also checked against the SRS and Hazard Analysis.

8 Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

   Our SRS and Hazard Analysis documents that identified several functional, non-functional, and safety requirements gave us a good starting point for defining the test plan. We used these documents to work from a specific requirement to an input to a test, an expected result, and a method to check that result, without having the need to come up with a general list of tests beforehand. For example, the requirements for stale health data and bounded recovery were used to specify tests for ensuring correctness during delayed health reports or a follower node getting interrupted.

   We also divided the document by responsibility to improve parallelism and reduce blockers while our team worked. Dividing the document and reviewing things together was important as the verification plan, test cases, traceability tables and symbolic parameters all described related activities, just from different perspectives.

   Compared to the SRS and Hazard Analysis documents, we clarified which members owned the sections earlier, as well as thoroughly reviewed and discussed the drafts and pull requests before merging. This helped align the different parts of the V&V, such as test cases and symbolic parameters before merging them together.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   A difficult part of the deliverable was planning the tests for the distributed behavior of Lake (i.e. testing how Lake behaves across computers with different system configurations and hardware capabilities), before the implementation and hardware behaviour were available. We addressed this by separating lightweight automated CI tests from hardware and system tests we would run on our own hardware.

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

   - **Jinwoo Hong:** Evaluating coverage and memory-checking tools for Rust, especially for areas involving ggml through an unsafe foreign function interface. This is important for testing as it requires figuring out what can and cannot be determined correctness-wise in this area.
   - **Gary Qin:** Designing experiments to determine model capacity, max concurrent requests, and output quality and compare with past baselines, while also configuring various model settings and device configurations.
   - **Matthew Nesbitt:** Building CI workflows across Linux, Windows, and macOS operating systems, while separating automated CI tests from those that require real manual hardware configurations.
   - **Colin Chambachan:** Implementing various fixtures and fault injection tests that can reproduce failures in the distributed system while keeping these tests automated in the CI.
   - **Ethan Walsh:** Ensuring tests are traceable to requirements and being able to audit and inspect missed acceptance criteria, infeasible tests, and any other changes that require tests to be updated.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge

or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

- **Jinwoo Hong:**
  - **Approach 1:** Study the documentation for how to utilize the coverage and memory checking tools for Rust. Read Section 20.1 of the Rust Book to learn about calling C functions and creating safe abstractions around unsafe Rust.
  - **Approach 2:** Compare and learn the tools by working on a small example of both Rust and C communicating.
  - **Chosen approach and reason:** The best approach combines both, focusing on reading documentation and also trying a small implementation to get a better understanding of potential difficulties when implementing with these tools.
- **Gary Qin:**
  - **Approach 1:** Review how existing benchmarks work for evaluating LLMs (e.g., MMLU) and study their methodologies.
  - **Approach 2:** Build a small experiment that varies one factor at a time to easily determine how different changes affect end results.
  - **Chosen approach and reason:** The proposed approach was to run a small experiment supported by documentation and methods used in other benchmarks. This can reveal which configurations cause unstable results before full hardware runs.
- **Matthew Nesbitt:**
  - **Approach 1:** Study the documentation for GitHub Actions and how Rust and Python workflows work across Linux, Windows, and macOS, focusing on areas such as dependency installation, caching, and retaining test reports so that failed runs can be investigated.
  - **Approach 2:** Build a small workflow that compiles a component and runs a test on it on one operating system. Also introduce a failing test to see how the error information can be retrieved.

  - **Chosen approach and reason:** The second approach was chosen because it allows for incrementally understanding and building the CI workflows. Once small components are built, they can be extended to the other operating systems to check platform-specific build and test behaviour.

- **Colin Chambachan:**
  - **Approach 1:** Review various examples of test fixtures, test doubles, and any course content from SFWRENG 3S03 that may be relevant for this area.
  - **Approach 2:** Implement a small test for fault injection, such as withholding a node's health reports until they become stale, and ensuring the system behaves correctly according to the relevant requirements.
  - **Chosen approach and reason:** Approach 1 should be a prerequisite if knowledge on the material is insufficient, otherwise approach 2 was chosen as it allows for more incremental improvements and understanding of the system and tests.

- **Ethan Walsh:**
  - **Approach 1:** Review the course V&V checklist and the team's traceability table to learn how to identify and interpret missing tests or unclear results.
  - **Approach 2:** Review some requirements and tests with a teammate then compare each other's findings about any missing cases or infeasible test procedures.
  - **Chosen approach and reason:** Assuming familiarity with the course material, the second approach was chosen for helping ensure there are no overlooked results, and that it better applies to Lake's use-case.